You should see the id values in the response without needing to query for them.

Fortunately Active Record supports RETURNING and you can experiment with this using bin/rails console. Insert a single Driver using .insert_all() and ask for the id column value:

ruby/insert_all_with_returning.rb

```
Driver.insert_all([
  {
    first_name: 'Andrew',
    last_name: 'Atkinson',
    email: "andrew.atkinson@example.com",
    password_digest: SecureRandom.hex
  }],
  returning: [:id]
)

_.first["id"]
```

You've now used RETURNING to eliminate a query.

How else might you speed up queries?

Bounding Query Results Using LIMIT

Another common way to resolve a poorly performing query is to add boundary conditions. Bounds can be added by filtering to fewer rows or by adding a LIMIT to the amount of rows that are fetched at one time using the Active Record .limit() method.

Most queries should be bounded when they are first written, but for small row counts unbounded queries will typically perform OK, and then slow down or not run at a certain row count size. The tipping point will depend on the queries and the server resources.

Another way to add boundaries is to use the Active Record batch operations like .find_each(), .find_in_batches(), and .in_batches() (See: *Rails — find_each v.s find_in_batches v.s in_batches*[10]).

These methods are typically used to reduce the amount of memory consumed at once, since each result row turns into an Active Record instance. However these methods also add limits to queries that can help them run reliably even as row counts increase.

10. https://medium.com/lynns-dev-blog/rails-find-each-v-s-find-in-batches-v-s-in-batches-d5ca9bfe37d